

DOCKER

Multi-Web Server Deployment on AWS

Amazon EC2 • Amazon Linux • Docker • Apache HTTPD • Nginx

Project Overview	Details
Project Goal	Deploy and manage multiple containerized web servers on AWS and publish reusable custom images.
Infrastructure	Amazon EC2 running Amazon Linux with Docker Engine.
Containers	6 total - 3 Apache HTTPD + 3 Nginx containers.
Website Deployment	Villa Agency on Apache (httpd1:8081) and LUGX Gaming on Nginx (nginx1:8084).
Image Creation	Custom images created as villa-apache:v1 and lugx-nginx:v1.
Docker Hub	Both custom images published to public Docker Hub repositories.
Amazon ECR	Both v1 images published to private Amazon ECR repositories using IAM-based EC2 access.
Verification	Villa ECR image redeployed as a new container and verified successfully on port 8091.

Hands-on containerization project documentation

Project Architecture

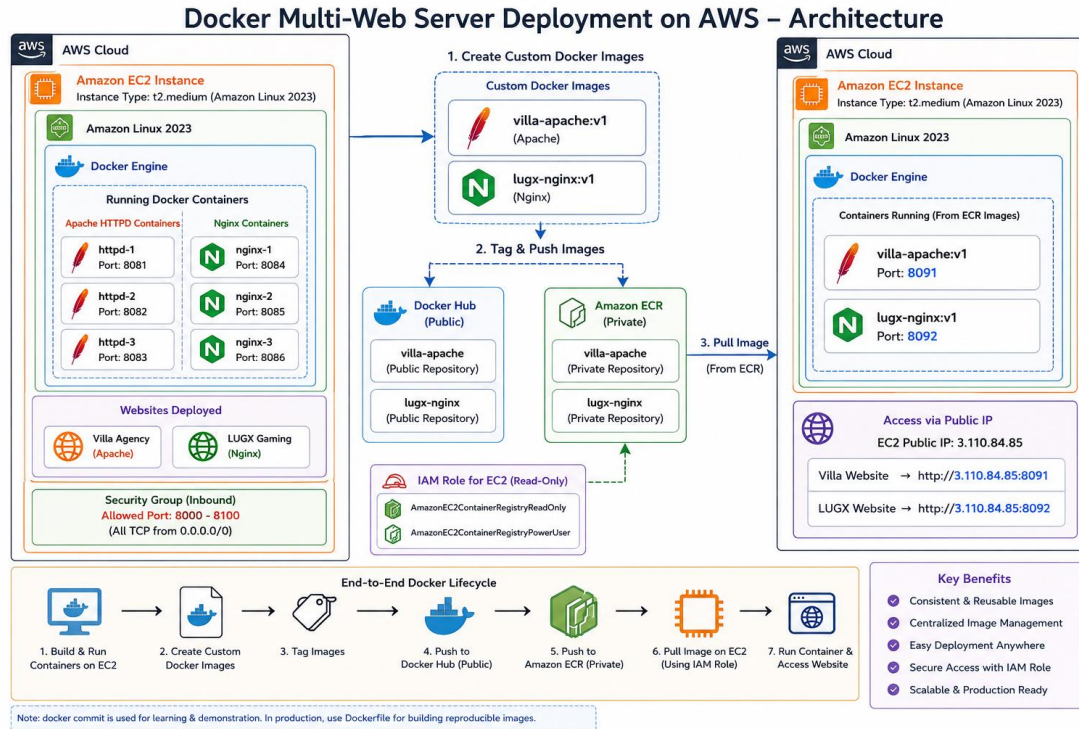


Figure 1 - End-to-end Docker deployment workflow: EC2 containers, custom images, Docker Hub, Amazon ECR, and website access.

About the Architecture

This architecture uses one Amazon Linux EC2 instance as the Docker host. Docker Engine runs six isolated web-server containers on the same instance: three Apache HTTPD containers and three Nginx containers. Host-to-container port mapping allows each web server to be accessed independently, while the EC2 security group permits the project web traffic through the configured 8000-8100 port range.

Two complete website templates were deployed to demonstrate real application hosting rather than only default web-server pages. The Villa Agency website runs inside the Apache container, while the LUGX Gaming website runs inside the Nginx container. This shows how different web-server technologies and website workloads can run side by side on a single EC2 host without requiring a separate virtual machine for every server.

After the websites were verified, the configured containers were converted into reusable versioned images named villa-apache:v1 and luxg-nginx:v1. These images were tagged and published to Docker Hub as public repositories and to Amazon ECR as private AWS repositories. The EC2 instance used an IAM role for ECR access, avoiding long-term AWS credentials on the server. Finally, the Villa image stored in ECR was used to launch a new container on port 8091, confirming the complete lifecycle from container deployment to image creation, registry storage, and successful redeployment.

1. What Is Docker?

Docker is a containerization platform used to package an application together with the files and dependencies it needs and run it as an isolated container. Containers share the host operating system kernel, which makes them generally lighter and faster to start than full virtual machines.

In this project, one Amazon Linux EC2 virtual machine acts as the Docker host. Instead of launching six separate EC2 instances for six web servers, six isolated web-server containers run on the same EC2 instance.

2. Why Containers?

- Isolation: each container has its own processes and filesystem view.
- Portability: the same image can be used consistently across environments.
- Efficiency: multiple containers can share one host operating system.
- Fast startup: containers usually start much faster than full virtual machines.
- Repeatability: images provide a reusable blueprint for creating containers.

3. Virtual Machine vs Container

A virtual machine virtualizes hardware and normally includes a complete guest operating system. A container virtualizes at the operating-system level and shares the host kernel. The comparison below is useful as a high-level learning reference; exact performance and security depend on the workload and configuration.

<i>Virtual Machine</i>	<i>Container</i>
• Heavyweight	• Lightweight
• Limited performance	• Native performance
• Each VM runs in its own OS	• All containers share the host OS
• Hardware-level virtualization	• OS virtualization
• Startup time in minutes	• Startup time in milliseconds
• Allocates required memory	• Requires less memory space
• Fully isolated and hence more secure	• Process-level isolation, possibly less secure

Figure 2 - High-level Virtual Machine vs Container comparison

4. Docker Concepts You Should Know

Concept	Meaning
Docker Engine	The software running on the host that builds and runs containers.
Image	A read-only template used to create containers, for example httpd or nginx.
Container	A running instance of an image.
Registry	A place where images are stored and distributed, such as Docker Hub.
Port Mapping	Maps a host port to a container port, for example 8081:80.
Volume / Bind Mount	Provides persistent or host-based storage to a container.
Dockerfile	A text file containing instructions for building a custom image.
Docker Compose	A declarative way to define and run multiple related containers.

5. AWS Infrastructure

The Docker host is an Amazon Linux EC2 instance in the Mumbai Region. The EC2 instance provides the compute, while the Security Group controls network access to SSH and the mapped web ports.

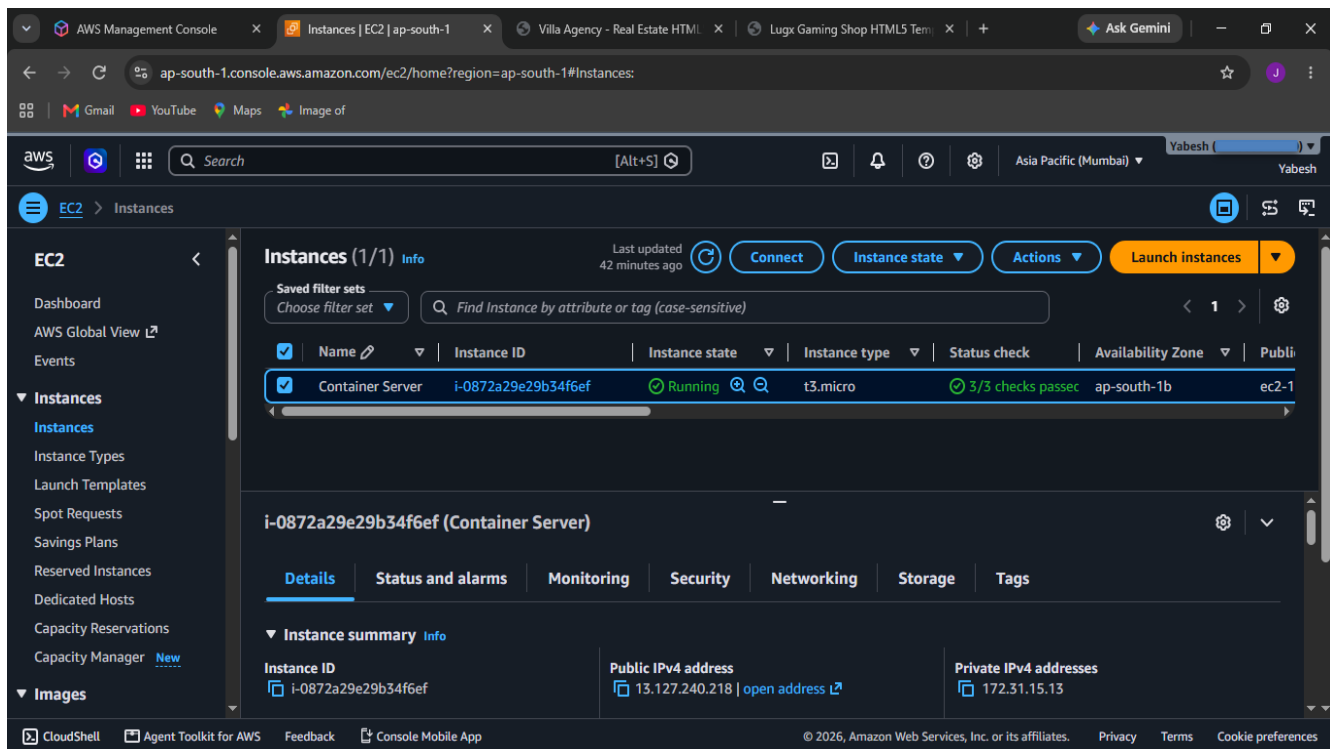


Figure 3 - Amazon EC2 instance used as the Docker host

For the lab, a Custom TCP range covering the Docker host ports was configured. In production, expose only the ports that are required and restrict administrative access such as SSH to trusted IP addresses.

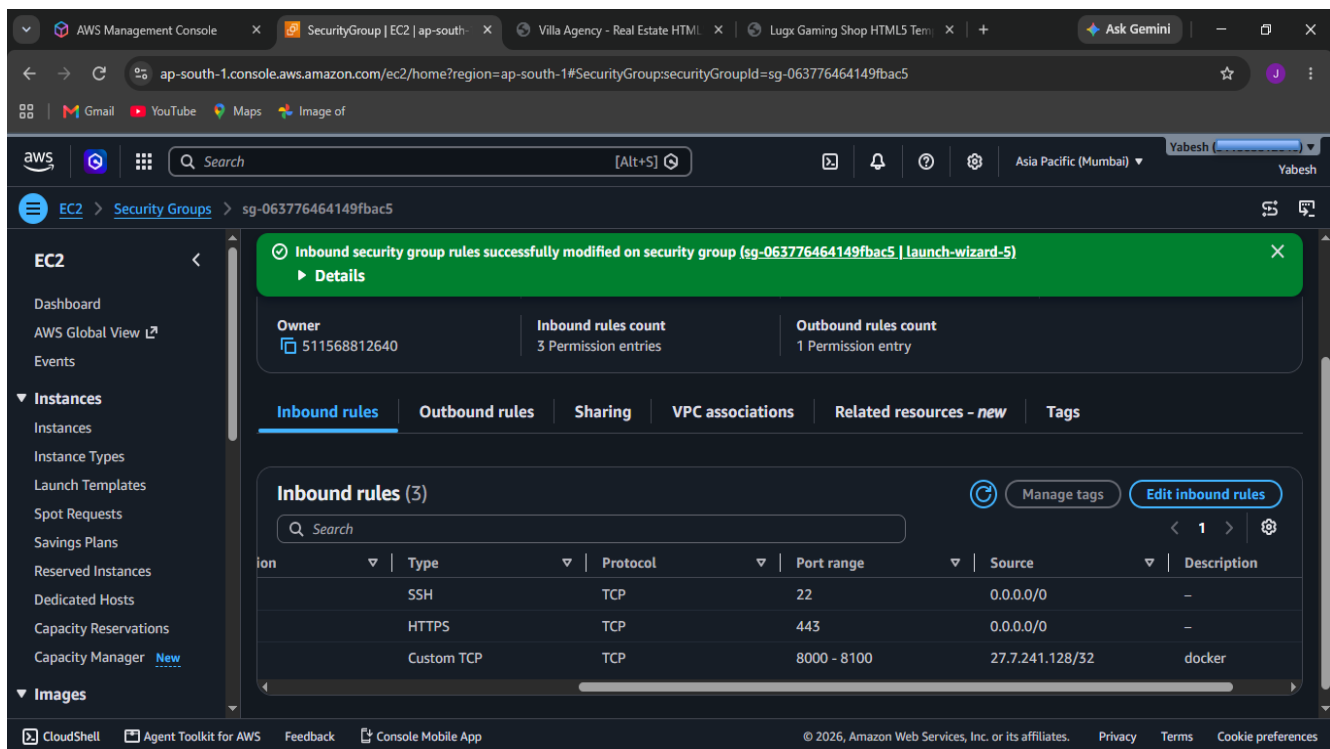


Figure 4 - Security Group rule allowing the lab web-server port range

6. Installing Docker on Amazon Linux

```
sudo yum update -y
sudo yum install -y docker
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
newgrp docker
docker --version
```

The Docker service is started and enabled so the Docker daemon is available. Adding ec2-user to the docker group allows Docker commands to be executed without prefixing every command with sudo.

7. Images: The Blueprint

Two official web-server images were used:

```
docker pull httpd
docker pull nginx
docker images
```

Pulling an image downloads the reusable application layers. An image itself is not the running web server; a container is created from that image.

8. Containers: Running the Images

Three Apache HTTPD containers:

```
docker run -d --name httpd1 -p 8081:80 httpd
docker run -d --name httpd2 -p 8082:80 httpd
docker run -d --name httpd3 -p 8083:80 httpd
```

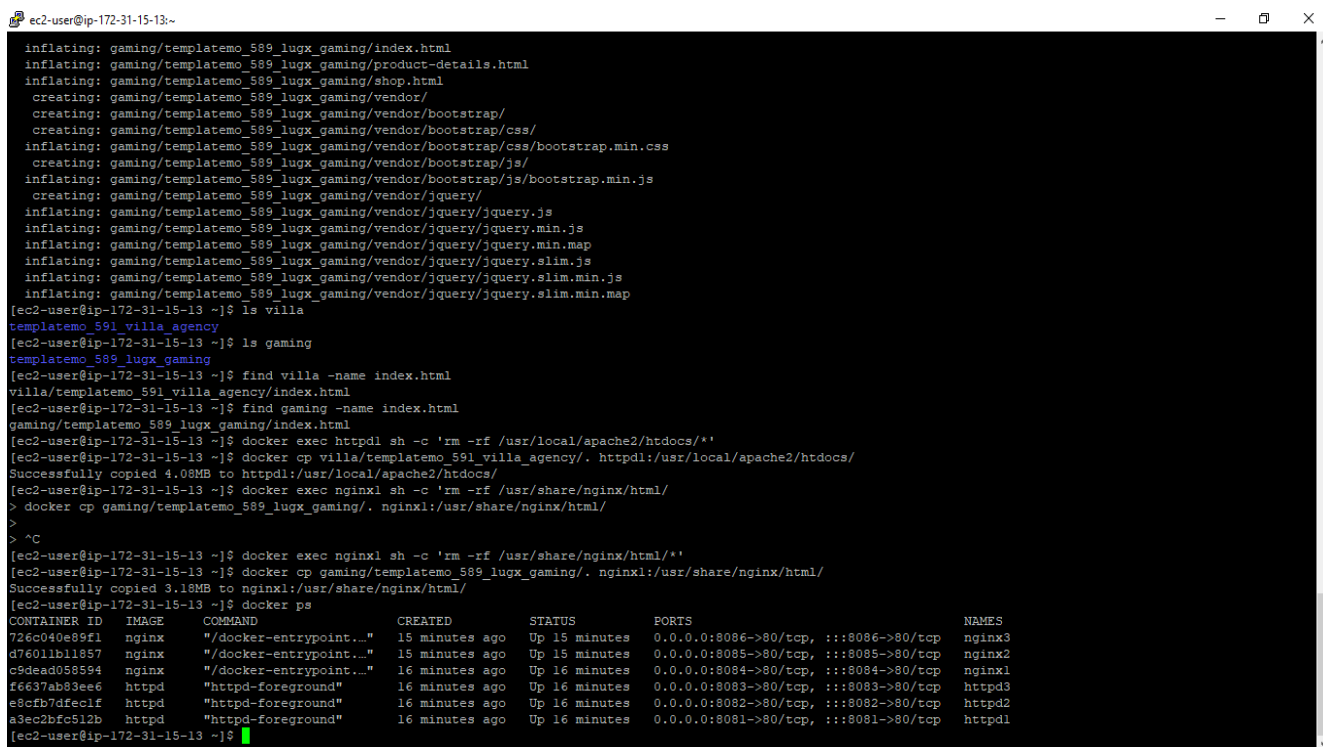
Three Nginx containers:

```
docker run -d --name nginx1 -p 8084:80 nginx
docker run -d --name nginx2 -p 8085:80 nginx
docker run -d --name nginx3 -p 8086:80 nginx
```

The -d option runs the container in detached mode, --name assigns a readable container name, and -p publishes a host port to port 80 inside the container.

10. Verifying All Six Containers

```
docker ps
docker ps -a
```



```
ec2-user@ip-172-31-15-13:~
inflating: gaming/templatemo_589_lugx_gaming/index.html
inflating: gaming/templatemo_589_lugx_gaming/product-details.html
inflating: gaming/templatemo_589_lugx_gaming/shop.html
creating: gaming/templatemo_589_lugx_gaming/vendor/
creating: gaming/templatemo_589_lugx_gaming/vendor/bootstrap/
creating: gaming/templatemo_589_lugx_gaming/vendor/bootstrap/css/
inflating: gaming/templatemo_589_lugx_gaming/vendor/bootstrap/css/bootstrap.min.css
creating: gaming/templatemo_589_lugx_gaming/vendor/bootstrap/js/
inflating: gaming/templatemo_589_lugx_gaming/vendor/bootstrap/js/bootstrap.min.js
creating: gaming/templatemo_589_lugx_gaming/vendor/jquery/
inflating: gaming/templatemo_589_lugx_gaming/vendor/jquery/jquery.js
inflating: gaming/templatemo_589_lugx_gaming/vendor/jquery/jquery.min.js
inflating: gaming/templatemo_589_lugx_gaming/vendor/jquery/jquery.min.map
inflating: gaming/templatemo_589_lugx_gaming/vendor/jquery/jquery.slim.js
inflating: gaming/templatemo_589_lugx_gaming/vendor/jquery/jquery.slim.min.js
inflating: gaming/templatemo_589_lugx_gaming/vendor/jquery/jquery.slim.min.map
[ec2-user@ip-172-31-15-13 ~]$ ls villa
templatemo_591_villa_agency
[ec2-user@ip-172-31-15-13 ~]$ ls gaming
templatemo_589_lugx_gaming
[ec2-user@ip-172-31-15-13 ~]$ find villa -name index.html
villa/templatemo_591_villa_agency/index.html
[ec2-user@ip-172-31-15-13 ~]$ find gaming -name index.html
gaming/templatemo_589_lugx_gaming/index.html
[ec2-user@ip-172-31-15-13 ~]$ docker exec httpd1 sh -c 'rm -rf /usr/local/apache2/htdocs/'
[ec2-user@ip-172-31-15-13 ~]$ docker cp villa/templatemo_591_villa_agency/. httpd1:/usr/local/apache2/htdocs/
Successfully copied 4.08MB to httpd1:/usr/local/apache2/htdocs/
[ec2-user@ip-172-31-15-13 ~]$ docker exec nginx1 sh -c 'rm -rf /usr/share/nginx/html/'
[ec2-user@ip-172-31-15-13 ~]$ docker cp gaming/templatemo_589_lugx_gaming/. nginx1:/usr/share/nginx/html/
^C
[ec2-user@ip-172-31-15-13 ~]$ docker exec nginx1 sh -c 'rm -rf /usr/share/nginx/html/'
[ec2-user@ip-172-31-15-13 ~]$ docker cp gaming/templatemo_589_lugx_gaming/. nginx1:/usr/share/nginx/html/
Successfully copied 3.18MB to nginx1:/usr/share/nginx/html/
[ec2-user@ip-172-31-15-13 ~]$ docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED            STATUS              PORTS                               NAMES
726c040e89f1      nginx              "/docker-entrypoint..." 15 minutes ago    Up 15 minutes      0.0.0.0:8086->80/tcp, :::8086->80/tcp nginx3
d7601b1b1857      nginx              "/docker-entrypoint..." 15 minutes ago    Up 15 minutes      0.0.0.0:8085->80/tcp, :::8085->80/tcp nginx2
c9dead058594      nginx              "/docker-entrypoint..." 16 minutes ago    Up 16 minutes      0.0.0.0:8084->80/tcp, :::8084->80/tcp nginx1
f6637ab83ee6      httpd              "httpd-foreground"       16 minutes ago    Up 16 minutes      0.0.0.0:8083->80/tcp, :::8083->80/tcp httpd3
e8cfb7dfecl1f     httpd              "httpd-foreground"       16 minutes ago    Up 16 minutes      0.0.0.0:8082->80/tcp, :::8082->80/tcp httpd2
a3ec2bfc512b      httpd              "httpd-foreground"       16 minutes ago    Up 16 minutes      0.0.0.0:8081->80/tcp, :::8081->80/tcp httpd1
[ec2-user@ip-172-31-15-13 ~]$
```

Figure 4 - PuTTY terminal showing six running containers and their port mappings

The docker ps output confirms that httpd1-httpd3 and nginx1-nginx3 are running. Each uses a different host port, so all six web servers can coexist on one EC2 instance.

11. Deploying Real Website Templates

To demonstrate a practical workload, two complete static website templates were downloaded. Static templates contain files such as HTML, CSS, JavaScript, fonts, and images and can be served directly by Apache or Nginx.

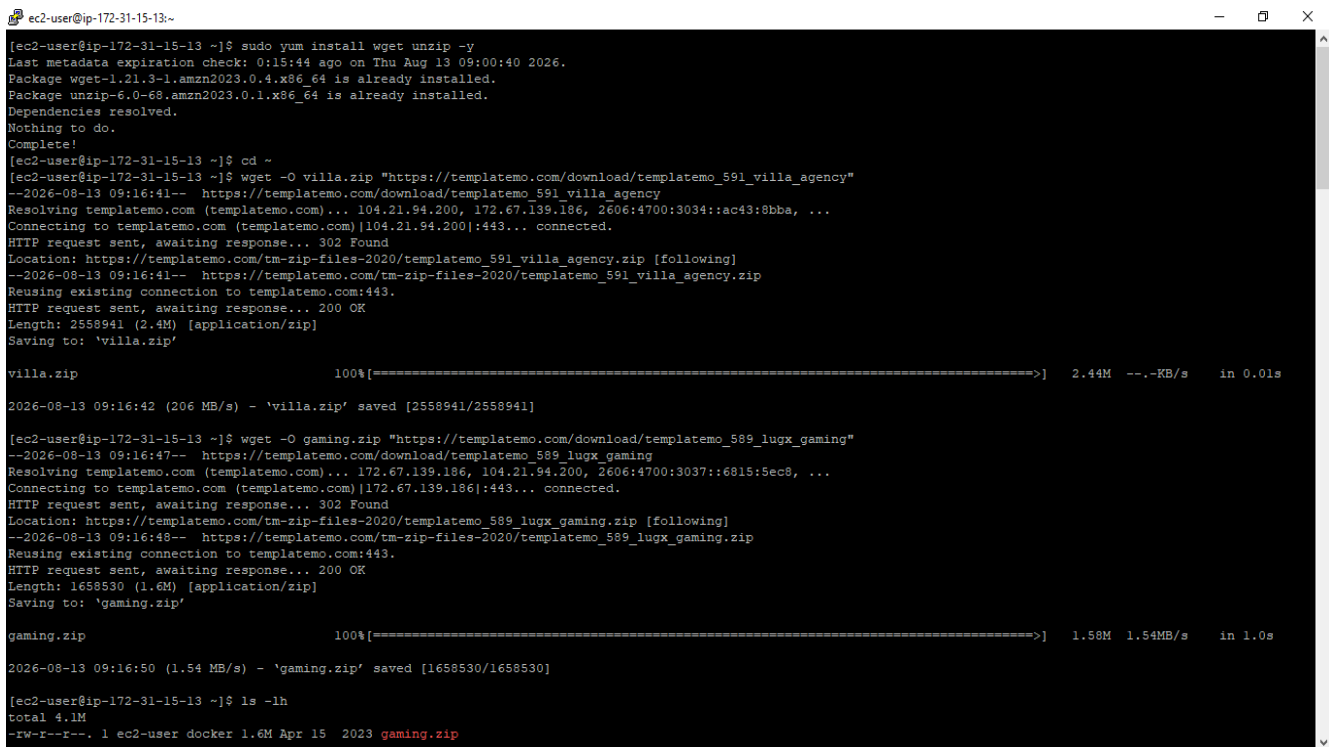
11.1 Download and Extract

```
sudo yum install -y wget unzip
cd ~
```

```
wget -O villa.zip "https://templatemo.com/download/templatemo_591_villa_agency"
wget -O gaming.zip "https://templatemo.com/download/templatemo_589_lugx_gaming"
```

```
mkdir villa gaming
unzip villa.zip -d villa
unzip gaming.zip -d gaming
```

```
find villa -name index.html
find gaming -name index.html
```

A terminal window screenshot showing the execution of commands to install wget and unzip, download two website templates (villa.zip and gaming.zip) from templatemo.com, and then list the files in the current directory. The terminal output shows the successful installation of packages, the download progress for both files, and the final directory listing showing the downloaded files and their sizes.

```
ec2-user@ip-172-31-15-13:~$ sudo yum install wget unzip -y
Last metadata expiration check: 0:15:44 ago on Thu Aug 13 09:00:40 2026.
Package wget-1.21.3-1.amzn2023.0.4.x86_64 is already installed.
Package unzip-6.0-68.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
ec2-user@ip-172-31-15-13 ~]$ cd ~
ec2-user@ip-172-31-15-13 ~]$ wget -O villa.zip "https://templatemo.com/download/templatemo_591_villa_agency"
--2026-08-13 09:16:41-- https://templatemo.com/download/templatemo_591_villa_agency
Resolving templatemo.com (templatemo.com)... 104.21.94.200, 172.67.139.186, 2606:4700:3034::ac43:8bba, ...
Connecting to templatemo.com (templatemo.com)|104.21.94.200|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://templatemo.com/tm-zip-files-2020/templatemo_591_villa_agency.zip [following]
--2026-08-13 09:16:41-- https://templatemo.com/tm-zip-files-2020/templatemo_591_villa_agency.zip
Reusing existing connection to templatemo.com:443.
HTTP request sent, awaiting response... 200 OK
Length: 2558941 (2.4M) [application/zip]
Saving to: 'villa.zip'

villa.zip                               100%[=====>] 2.44M --.-KB/s in 0.01s

2026-08-13 09:16:42 (206 MB/s) - 'villa.zip' saved [2558941/2558941]

ec2-user@ip-172-31-15-13 ~]$ wget -O gaming.zip "https://templatemo.com/download/templatemo_589_lugx_gaming"
--2026-08-13 09:16:47-- https://templatemo.com/download/templatemo_589_lugx_gaming
Resolving templatemo.com (templatemo.com)... 172.67.139.186, 104.21.94.200, 2606:4700:3037::6815:Sec8, ...
Connecting to templatemo.com (templatemo.com)|172.67.139.186|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://templatemo.com/tm-zip-files-2020/templatemo_589_lugx_gaming.zip [following]
--2026-08-13 09:16:48-- https://templatemo.com/tm-zip-files-2020/templatemo_589_lugx_gaming.zip
Reusing existing connection to templatemo.com:443.
HTTP request sent, awaiting response... 200 OK
Length: 1658530 (1.6M) [application/zip]
Saving to: 'gaming.zip'

gaming.zip                               100%[=====>] 1.58M 1.54MB/s in 1.0s

2026-08-13 09:16:50 (1.54 MB/s) - 'gaming.zip' saved [1658530/1658530]

ec2-user@ip-172-31-15-13 ~]$ ls -lh
total 4.1M
-rw-r--r-- 1 ec2-user docker 1.6M Apr 15 2023 gaming.zip
```

Figure 5 - Downloading the Villa Agency and LUGX Gaming templates

11.2 Apache Deployment - Villa Agency

The default Apache document root in the official httpd image is /usr/local/apache2/htdocs/. The default content was removed and the Villa Agency files were copied into httpd1.

```
docker exec httpd1 sh -c 'rm -rf /usr/local/apache2/htdocs/*'
```

```
docker cp villa/templatemo_591_villa_agency/. httpd1:/usr/local/apache2/htdocs/
```

Browser endpoint: `http://<EC2-PUBLIC-IP>:8081`

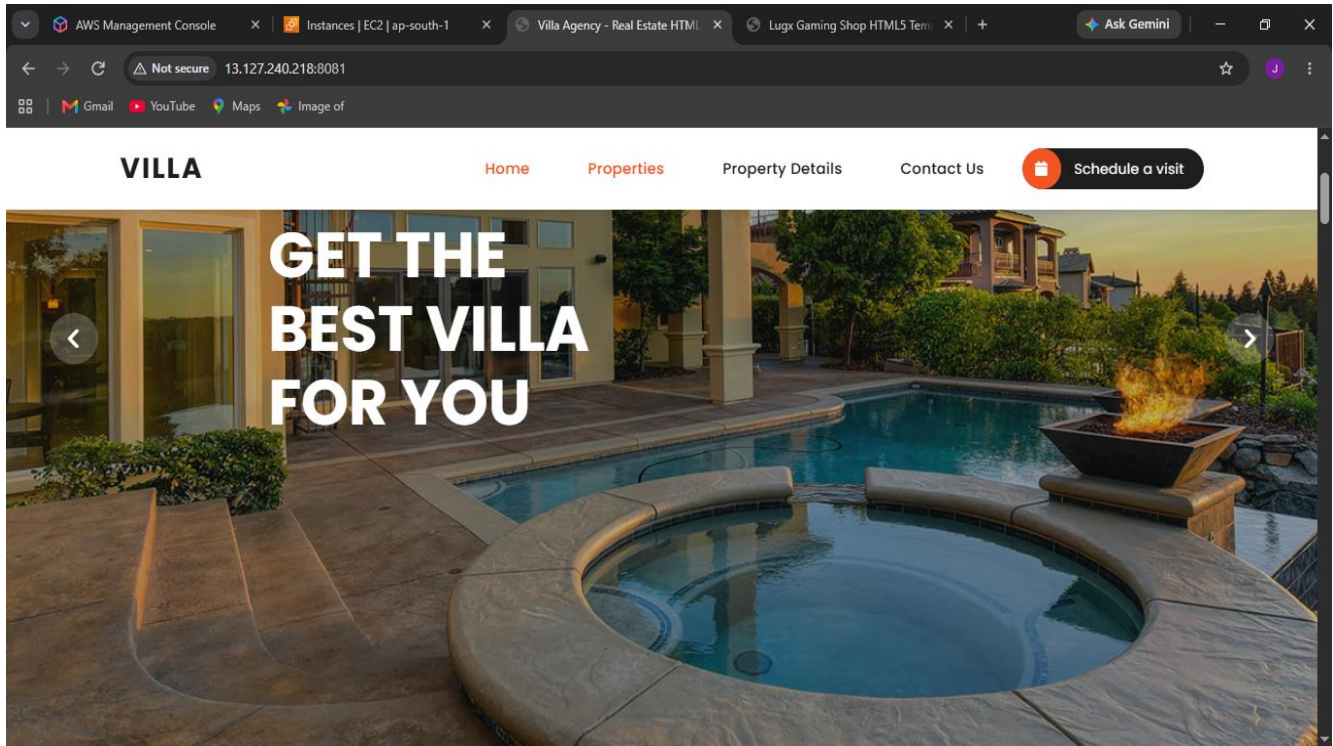


Figure 6 - Villa Agency successfully served by Apache HTTPD container httpd1 on port 8081

11.3 Nginx Deployment - LUGX Gaming

The default Nginx document root in the official image is /usr/share/nginx/html/. The default content was removed and the LUGX Gaming files were copied into nginx1.

```
docker exec nginx1 sh -c 'rm -rf /usr/share/nginx/html/*'
```

```
docker cp gaming/templatemo_589_lugx_gaming/. nginx1:/usr/share/nginx/html/
```

Browser endpoint: `http://<EC2-PUBLIC-IP>:8084`

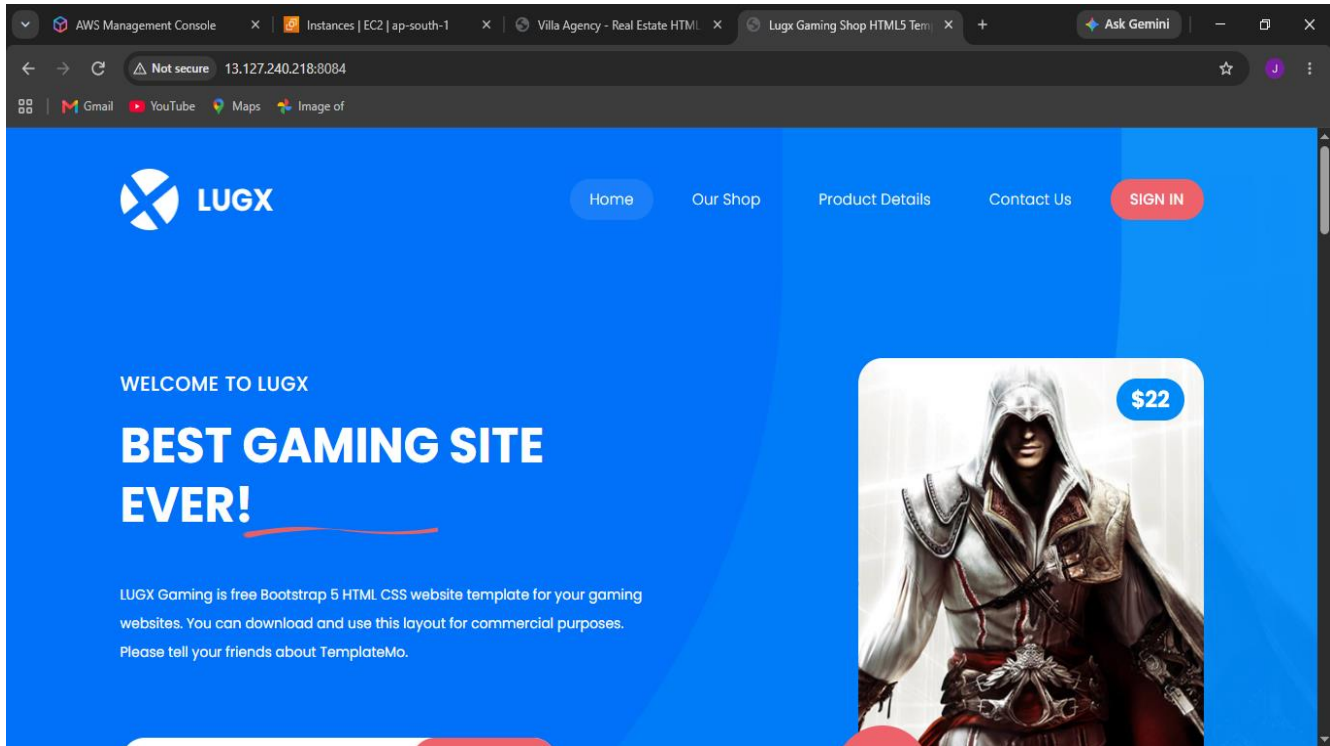


Figure 7 - LUGX Gaming successfully served by Nginx container nginx1 on port 8084

12. What Happens When a User Opens the Website?

1. The browser sends an HTTP request to the EC2 public IP and host port, for example port 8081.
2. The EC2 Security Group decides whether that inbound connection is allowed.
3. Docker receives traffic on host port 8081 and forwards it to port 80 inside httpd1.
4. Apache reads index.html and related CSS, JavaScript, and image files from its document root.
5. The response travels back to the browser and the complete website is rendered.

13. Container Lifecycle and Troubleshooting Commands

Command	Purpose
<code>docker ps</code>	Show running containers
<code>docker ps -a</code>	Show running and stopped containers
<code>docker images</code>	Show local images
<code>docker logs <name></code>	Read a container's logs
<code>docker exec -it <name> sh</code>	Open a shell inside a container
<code>docker stop <name></code>	Stop a container
<code>docker start <name></code>	Start a stopped container
<code>docker restart <name></code>	Restart a container
<code>docker rm <name></code>	Remove a stopped container
<code>docker rmi <image></code>	Remove an unused image

14. Image vs Container - The Key Difference

An image is the blueprint; a container is a running instance of that blueprint. The same httpd image was used to create three independent Apache containers, and the same nginx image was used to create three independent Nginx containers. This is one of the most important Docker concepts.

15. Storage and Persistence

In this lab, website files were copied directly into running containers with `docker cp`. This is easy for learning, but container writable data is not the preferred place for persistent application content. In a production-style workflow, use a bind mount or Docker volume, or build the website into a custom Docker image using a Dockerfile.

```
# Example bind mount concept
```

```
docker run -d -p 8081:80 -v /home/ec2-user/site:/usr/local/apache2/htdocs:ro httpd
```

16. Dockerfile - Next Step

A Dockerfile makes deployment reproducible by creating a custom image that already contains the website.

```
FROM httpd:latest
```

```
COPY ./website/ /usr/local/apache2/htdocs/
```

Typical build and run flow:

```
docker build -t villa-site .
```

```
docker run -d --name villa -p 8081:80 villa-site
```

17. Docker Compose - Managing Multiple Containers

For larger multi-container setups, Docker Compose can define containers, images, ports, volumes, and networks in one YAML file. Instead of manually running many docker run commands, the environment can be started with docker compose up -d.

18. Docker Networking

Docker provides networking so containers can communicate while remaining isolated from unrelated workloads. In this lab, published host ports make selected container services reachable from outside the EC2 instance. For application stacks, user-defined Docker networks can allow containers to communicate by container name without exposing every internal service publicly.

19. Security Considerations

- Do not expose unnecessary container ports to the internet.
- Restrict SSH access to trusted IP addresses.
- Use maintained images and update them regularly.
- Avoid storing passwords, API keys, or secrets directly in images.
- Run applications with the least privileges required.
- Use HTTPS and a reverse proxy/load balancer for internet-facing production workloads.
- Treat public IP addresses, account identifiers, and infrastructure screenshots carefully when publishing documentation.

20. What This Project Demonstrates

- Provisioning an Amazon Linux EC2 host for container workloads.
- Installing and operating Docker Engine.
- Pulling Apache HTTPD and Nginx images.
- Creating six independent containers from two reusable images.
- Understanding detached mode, container naming, and host-to-container port mapping.
- Configuring AWS Security Group access for containerized web services.
- Deploying full static website templates into Apache and Nginx document roots.
- Validating container status and testing applications from a browser.
- Understanding the path from browser -> Security Group -> EC2 host port -> Docker -> container port -> web server.

21. Production Improvements

This is intentionally a learning lab. A production design would normally avoid exposing many high-numbered ports directly. Common improvements include custom Docker images, persistent storage where needed, container health checks, centralized logging, HTTPS, a reverse proxy or load balancer, automated deployment, vulnerability scanning, and an orchestration platform such as Amazon ECS or Amazon EKS when the workload requires it.

22. Final Result

The project successfully runs six web-server containers on a single Amazon Linux EC2 instance. Three containers use Apache HTTPD and three use Nginx. Villa Agency is deployed on httpd1 through host port 8081, while LUGX Gaming is deployed on nginx1 through host port 8084. The result demonstrates the complete Docker flow from image download to container creation, network exposure, application deployment, and browser validation.

23. Quick Docker Mental Model

Dockerfile -> docker build -> Image

|

v

docker run

|

v

Container

|

host port -> container port

|

v

Application/Web Server

If you remember one idea: an image is a reusable blueprint, a container is a running instance of that image, and port mapping is how an external client reaches a service running inside the container.

24. Container Image Creation and Registry Deployment

After validating the Apache and Nginx website containers, the project was extended to cover the next stage of the Docker lifecycle: converting the configured containers into reusable images, tagging those images, publishing them to Docker Hub and Amazon Elastic Container Registry (Amazon ECR), and finally launching a new container from the ECR image to verify that the application could be redeployed successfully.

24.1 Creating Custom Images from the Website Containers

The running httpd1 container already contained the Villa Agency website, while nginx1 contained the LUGX Gaming website. For this lab, docker commit was used to capture the filesystem state of each configured container as a new reusable image.

```
docker commit httpd1 villa-apache:v1
docker commit nginx1 lugx-nginx:v1
```

```
docker images
```

This produced two custom images: villa-apache:v1 and lugx-nginx:v1. The v1 tag identifies the first version of each custom image.

24.2 Tagging Images for Container Registries

A Docker image can have multiple tags that point to the same underlying image. Registry-specific tags were added so the images could be pushed to Docker Hub and Amazon ECR.

```
# Docker Hub tags
docker tag villa-apache:v1 yabesh007/villa-apache:v1
docker tag lugx-nginx:v1 yabesh007/lugx-nginx:v1
```

```
# Amazon ECR tags
docker tag villa-apache:v1 511568812640.dkr.ecr.ap-south-1.amazonaws.com/villa-apache:v1
docker tag lugx-nginx:v1 511568812640.dkr.ecr.ap-south-1.amazonaws.com/lugx-nginx:v1
```

```
ec2-user@ip-172-31-15-13 ~$ docker images
REPOSITORY                                TAG                IMAGE ID            CREATED
511568812640.dkr.ecr.ap-south-1.amazonaws.com/lugx-nginx  v1                94b0e0162180       22 minutes ago
lugx-nginx                                v1                94b0e0162180       22 minutes ago
yabesh007/lugx-nginx                      v1                94b0e0162180       22 minutes ago
511568812640.dkr.ecr.ap-south-1.amazonaws.com/villa-apache  v1                00c46023277c       22 minutes ago
villa-apache                             v1                00c46023277c       22 minutes ago
yabesh007/villa-apache                    v1                00c46023277c       22 minutes ago
httpd                                     latest            46ababc59dbd       9 days ago
nginx                                     latest            5253dc86cc93       9 days ago

[ec2-user@ip-172-31-15-13 ~]$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
726c040e89f1   nginx    "/docker-entrypoint...." 23 hours ago  Up 32 minutes  0.0.0.0:8086->80/tcp,
:::8086->80/tcp   nginx3
d76011b11857   nginx    "/docker-entrypoint...." 23 hours ago  Up 32 minutes  0.0.0.0:8085->80/tcp,
:::8085->80/tcp   nginx2
c9dead058594   nginx    "/docker-entrypoint...." 23 hours ago  Up 32 minutes  0.0.0.0:8084->80/tcp,
:::8084->80/tcp   nginx1
f6637ab83ee6   httpd    "httpd-foreground"      23 hours ago  Up 32 minutes  0.0.0.0:8083->80/tcp,
:::8083->80/tcp   httpd3
e8cfb7dfec1f   httpd    "httpd-foreground"      23 hours ago  Up 32 minutes  0.0.0.0:8082->80/tcp,
:::8082->80/tcp   httpd2
a3ec2bfc512b   httpd    "httpd-foreground"      23 hours ago  Up 32 minutes  0.0.0.0:8081->80/tcp,
:::8081->80/tcp   httpd1
[ec2-user@ip-172-31-15-13 ~]$
```

Figure 8 - Docker images and running containers after creating Docker Hub and Amazon ECR tags

24.3 Publishing to Docker Hub

Docker Hub was used as the public image registry. After authenticating with Docker Hub, both images were pushed to public repositories so that they can be pulled by repository name and tag.

```
docker login
```

```
docker push yabesh007/villa-apache:v1
```

```
docker push yabesh007/lugx-nginx:v1
```

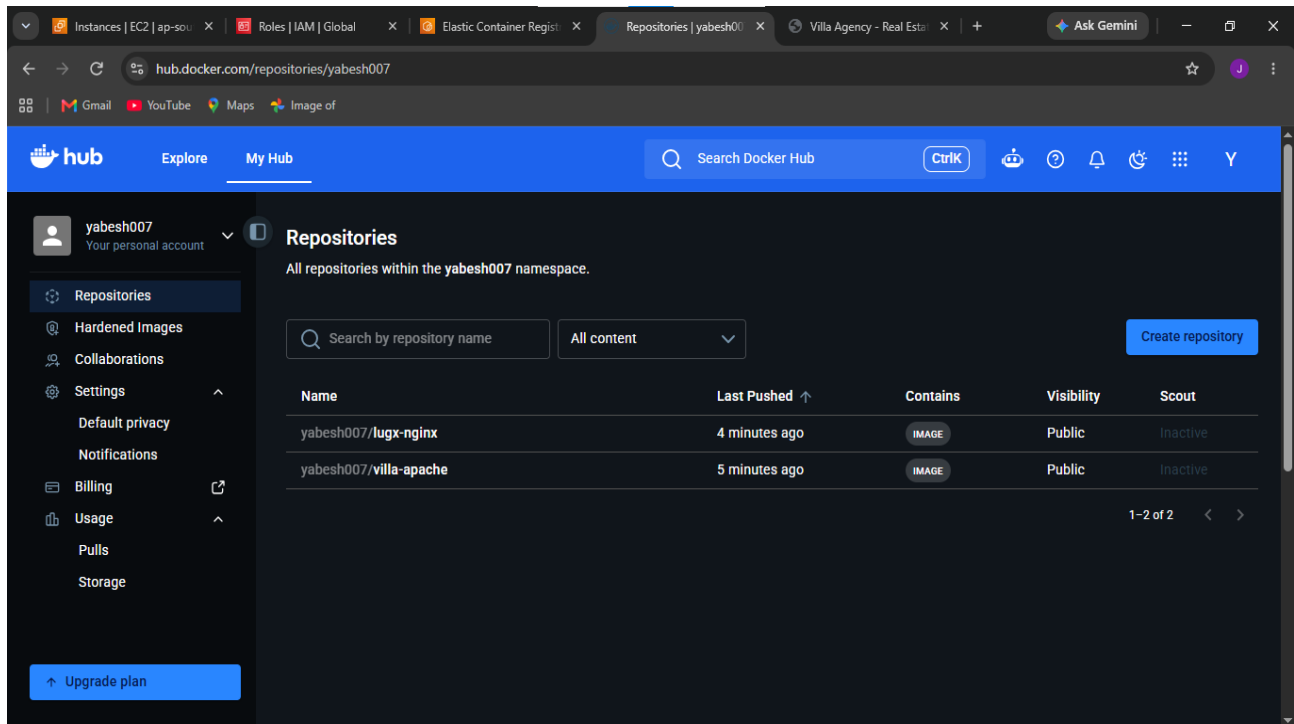


Figure 9 - Docker Hub showing the public LUGX Nginx and Villa Apache repositories

24.4 Amazon ECR Authentication with an EC2 IAM Role

Amazon ECR was used as the AWS-managed private container registry. Instead of storing long-term AWS access keys on the EC2 instance, an IAM role was attached to the EC2 instance to provide the required ECR permissions. Docker was then authenticated to the ECR registry using an authorization token generated by the AWS CLI.

```
aws sts get-caller-identity
```

```
aws ecr get-login-password --region ap-south-1 | \
docker login --username AWS --password-stdin \
511568812640.dkr.ecr.ap-south-1.amazonaws.com
```

24.5 Publishing the Images to Amazon ECR

Two private ECR repositories were created: villa-apache and lugx-nginx. The previously tagged images were then pushed to their corresponding repositories.

```
docker push 511568812640.dkr.ecr.ap-south-1.amazonaws.com/villa-apache:v1
```

```
docker push 511568812640.dkr.ecr.ap-south-1.amazonaws.com/lugx-nginx:v1
```

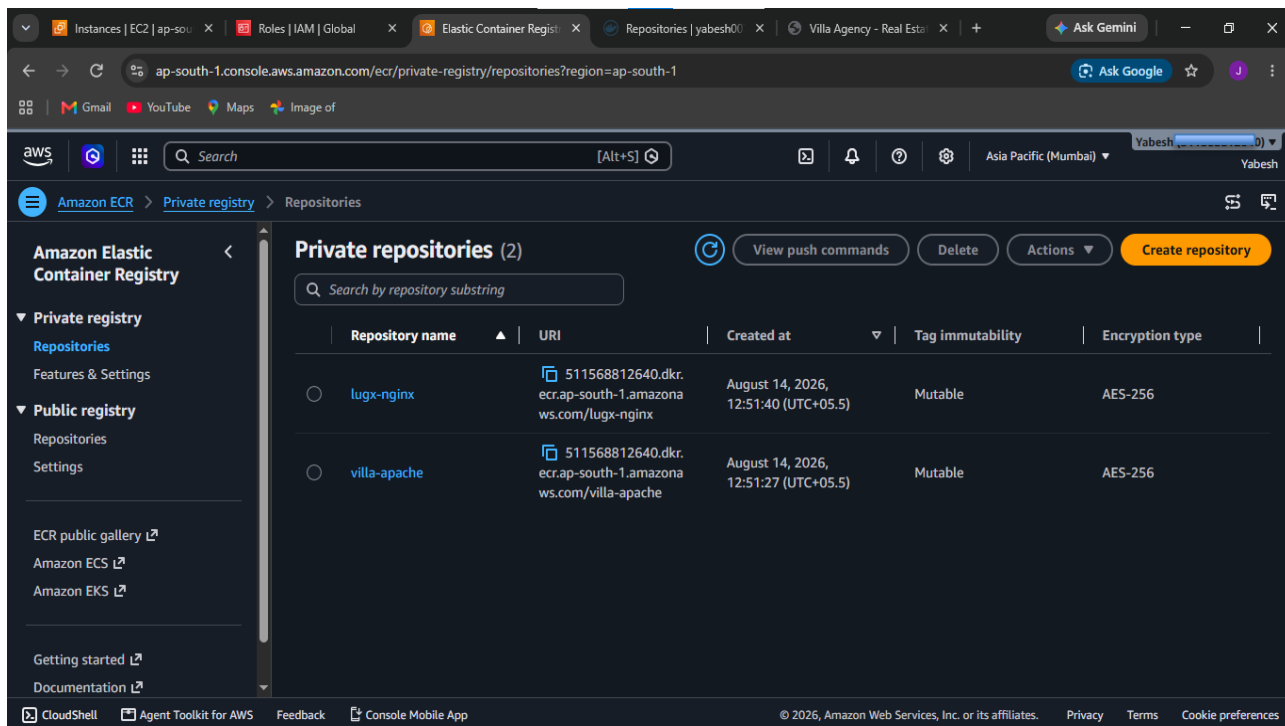


Figure 10 - Amazon ECR private registry containing the Villa Apache and LUGX Nginx repositories

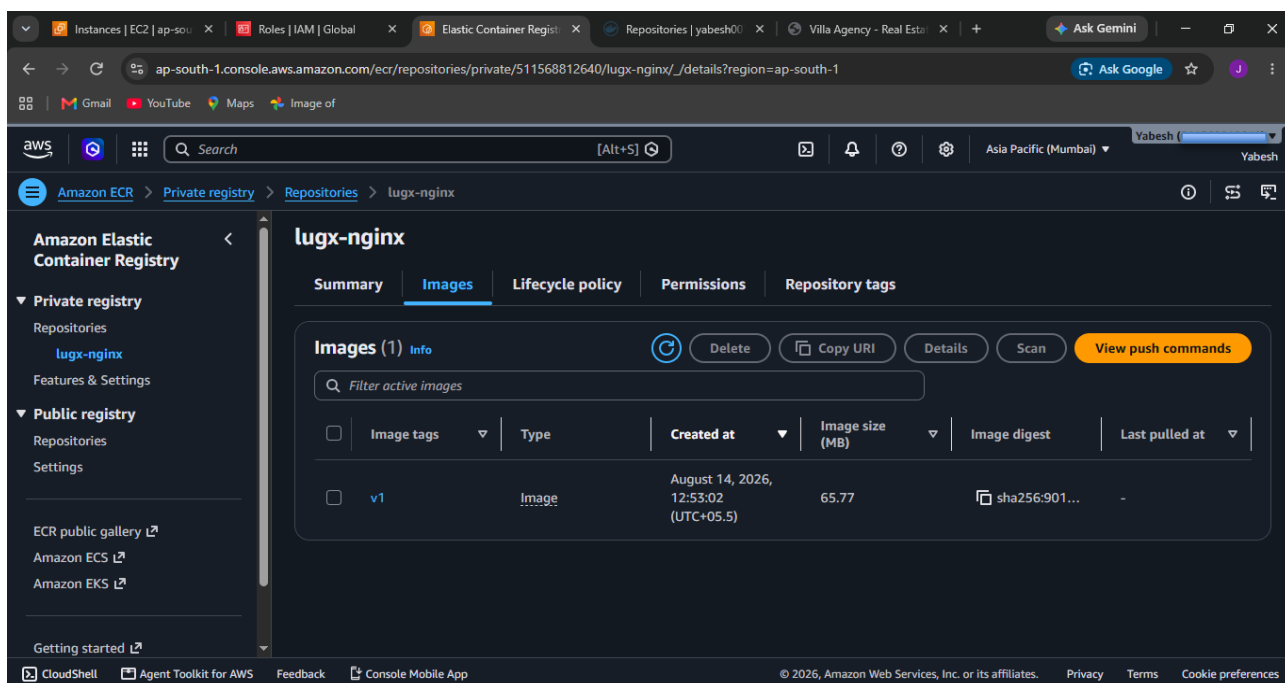


Figure 11 - LUGX Nginx image stored in Amazon ECR with the v1 tag

24.6 Redeploying an Image from Amazon ECR

Uploading an image to a registry proves that the image was stored, but a stronger verification is to launch a new container from the registry-tagged image. A separate test container was therefore created from the Villa Apache ECR image and mapped to host port 8091.

```
docker run -d --name ecr-villa-test -p 8091:80 \
511568812640.dkr.ecr.ap-southeast-1.amazonaws.com/villa-apache:v1
```

```
docker ps
```

The Villa Agency website loaded successfully through port 8091, verifying that the registry image was reusable and contained the expected website content.

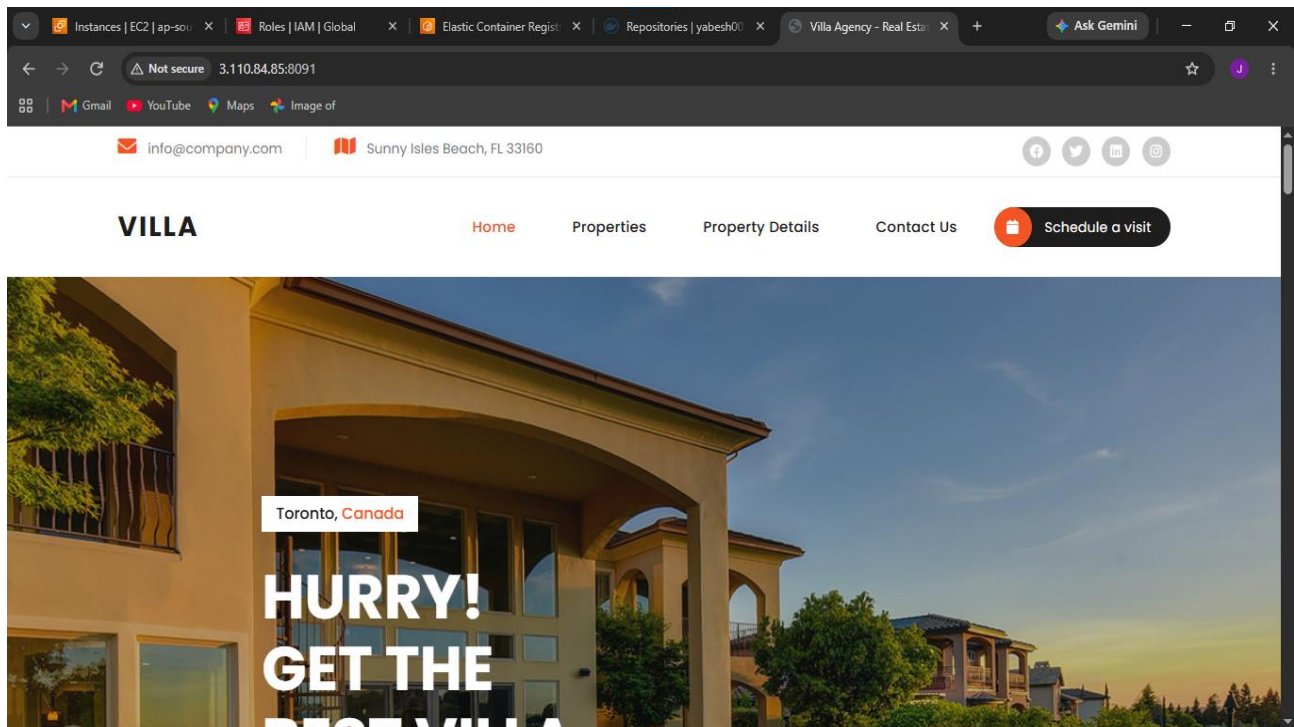
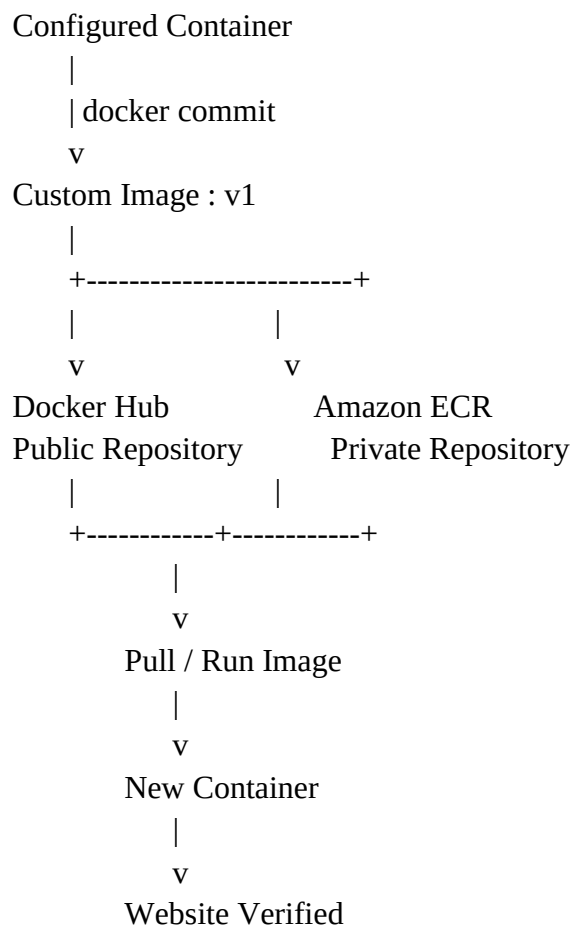


Figure 12 - Villa Agency successfully redeployed from the Amazon ECR image and verified on port 8091

24.7 Complete Image Lifecycle



24.8 What Was Learned from the Registry Extension

- How a configured running container can be captured as a custom image for a learning lab.
- How Docker tags identify image versions and registry destinations.
- How to authenticate and push images to Docker Hub.
- How public Docker Hub repositories differ from private Amazon ECR repositories.
- How an EC2 IAM role can provide AWS permissions without storing long-term access keys on the server.
- How to authenticate Docker to Amazon ECR using the AWS CLI.
- How to push custom images to ECR and verify the v1 tag in the repository.
- How to launch a new container from a registry image and verify the application in a browser.

24.9 Recommended Production Improvement

docker commit was useful here because it demonstrates the relationship between a configured container and an image. For a reproducible production workflow, the preferred approach is to define the application in a Dockerfile, build the image with docker build, version the image, and push that build artifact to the registry. This makes the image creation process repeatable and suitable for CI/CD automation.

25. Updated Final Project Outcome

The completed project now covers the Docker workflow from initial container deployment through image registry distribution and redeployment. Six web-server containers run on an Amazon Linux EC2 host, with Villa Agency served by Apache HTTPD and LUGX Gaming served by Nginx. The configured website containers were captured as versioned custom images, published to public Docker Hub repositories and private Amazon ECR repositories, and the Villa Apache ECR image was successfully used to launch an additional working container. This demonstrates Docker images, containers, port mapping, web-server deployment, image tagging, registry authentication, IAM-based ECR access, image push, and image reuse in one end-to-end AWS containerization project.

26. Complete Command Reference

The commands below summarize the main commands used throughout this project, from Docker installation and container deployment to website publishing, image creation, Docker Hub, Amazon ECR, and redeployment.

Docker Installation and Service

```
sudo yum update -y
sudo yum install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

Pull Base Images

```
docker pull httpd
docker pull nginx
docker images
```

Create the Six Web-Server Containers

```
docker run -d --name httpd1 -p 8081:80 httpd
docker run -d --name httpd2 -p 8082:80 httpd
docker run -d --name httpd3 -p 8083:80 httpd
docker run -d --name nginx1 -p 8084:80 nginx
docker run -d --name nginx2 -p 8085:80 nginx
docker run -d --name nginx3 -p 8086:80 nginx
docker ps
docker ps -a
```

Download and Deploy the Website Templates

```
wget https://templatemo.com/download/templatemo_591_villa_agency
wget https://templatemo.com/download/templatemo_589_lugx_gaming
unzip templatemo_591_villa_agency
unzip templatemo_589_lugx_gaming
docker cp templatemo_591_villa_agency/. httpd1:/usr/local/apache2/htdocs/
docker cp templatemo_589_lugx_gaming/. nginx1:/usr/share/nginx/html/
```

Container Management

```
docker stop httpd1 httpd2 httpd3 nginx1 nginx2 nginx3
docker start httpd1 httpd2 httpd3 nginx1 nginx2 nginx3
docker restart httpd1
docker logs httpd1
docker logs nginx1
docker exec -it httpd1 /bin/bash
docker exec -it nginx1 /bin/bash
```

Automatic Restart After EC2 Reboot

```
docker update --restart unless-stopped httpd1
docker update --restart unless-stopped httpd2
docker update --restart unless-stopped httpd3
docker update --restart unless-stopped nginx1
docker update --restart unless-stopped nginx2
docker update --restart unless-stopped nginx3
```

Create Custom Images

```
docker commit httpd1 villa-apache:v1
docker commit nginx1 lugx-nginx:v1
docker images
```

Docker Hub Login, Tag and Push

```
docker login
docker tag villa-apache:v1 yabesh007/villa-apache:v1
docker tag lugx-nginx:v1 yabesh007/lugx-nginx:v1
docker push yabesh007/villa-apache:v1
docker push yabesh007/lugx-nginx:v1
```

AWS and Amazon ECR Authentication

```
aws --version
aws sts get-caller-identity
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin
511568812640.dkr.ecr.ap-south-1.amazonaws.com
```

Tag Images for Amazon ECR

```
docker tag villa-apache:v1 511568812640.dkr.ecr.ap-south-1.amazonaws.com/villa-apache:v1
docker tag lugx-nginx:v1 511568812640.dkr.ecr.ap-south-1.amazonaws.com/lugx-nginx:v1
```

Push Images to Amazon ECR

```
docker push 511568812640.dkr.ecr.ap-south-1.amazonaws.com/villa-apache:v1
docker push 511568812640.dkr.ecr.ap-south-1.amazonaws.com/lugx-nginx:v1
```

Redeploy and Verify the ECR Image

```
docker run -d --name ecr-villa-test -p 8091:80 511568812640.dkr.ecr.ap-south-1.amazonaws.com/villa-  
apache:v1  
docker ps  
docker rm -f ecr-villa-test
```

Useful Cleanup and Inspection Commands

```
docker images  
docker ps  
docker ps -a  
docker rm <container-name>  
docker rmi <image-name>:<tag>  
clear
```